

STARDAM VALLEY

El Videojuego



Alicia Cámara Casares
acamarc02@educarex.es

2024/2025

ÍNDICE

1. OBJETIVO y RESULTADOS DE APRENDIZAJE	2
2. DESCRIPCIÓN DEL VIDEOJUEGO	2
2.1. Requisito funcional - Inicio de la aplicación	4
2.1.1. Nueva partida	4
2.1.2. Cargar partida	5
2.2. Requisito funcional - Programa principal	5
3. CONSEJOS Y CONSIDERACIONES	7
4. SOBRE EL PLAGIO O USO DE IA	8
5. ENTREGA	8
6. RÚBRICA DE CALIFICACIÓN	9

1. OBJETIVO y RESULTADOS DE APRENDIZAJE

Stardew Valley es un juego de simulación y gestión en el que el jugador asume el rol de un personaje que hereda una granja en mal estado y debe restaurarla y mantenerla. El objetivo principal es convertir esta granja en una próspera fuente de recursos, gestionando diferentes aspectos de la vida rural. Los jugadores pueden plantar y cosechar cultivos, regar sus plantas, y cuidar de animales como vacas, pollos y ovejas. El juego sigue un ciclo de estaciones (primavera, verano, otoño e invierno), lo que influye directamente en las actividades agrícolas y en los tipos de cultivos que pueden plantarse en cada época del año.

En este proyecto, desarrollaremos una versión simplificada llamada Stardam Valley, centrada exclusivamente en la mecánica de cultivo de cosechas y basada en los requisitos que se especifican en los posteriores apartados.

Esta actividad está orientada a evaluar la consecución de los siguientes resultados de aprendizaje del módulo de Acceso a datos:

- **RA1. Desarrolla aplicaciones que gestionan información almacenada en ficheros identificando el campo de aplicación de los mismos y utilizando clases específicas.**

Para ello se emplearán los criterios de evaluación asociados a ellos, tal y como se recogen en el RD405/2023 y supondrá el 35% de la calificación de los mismos.

2. DESCRIPCIÓN DEL VIDEOJUEGO

Nuestra **Granja** almacenará la información esencial para el correcto funcionamiento del juego:

- **Día actual del juego.** Al iniciar una nueva partida, el juego comenzará en el día 1. Este valor se incrementará conforme el jugador decida avanzar.
- **Estación del año.** La estación actual (PRIMAVERA, VERANO, OTOÑO e INVIERNO) influirá directamente en los tipos de cultivos que se pueden plantar y cosechar durante el juego. La duración de las estaciones se especifica en el fichero de propiedades, del que hablaremos en posteriores apartados.

- **Presupuesto.** Representa el dinero disponible del jugador, utilizado para la compra de semillas y otros recursos necesarios para mantener y expandir la granja.
- **Tienda.** La tienda es el lugar donde el jugador puede adquirir semillas. Cada día, ofrecerá una nueva combinación de semillas disponibles, limitadas a aquellas que sean adecuadas para la estación actual. Se asume que la tienda dispone de una cantidad ilimitada de cada tipo de semilla. La información sobre las semillas disponibles en el juego se encuentra almacenada en el archivo “semillas.xml”.
- **Almacén.** Almacena los frutos cosechados, manteniendo un registro de los frutos y las cantidades disponibles de cada uno de ellos para su posterior venta.

El **Huerto**, donde se plantarán las semillas adquiridas en la tienda, tendrá una estructura en forma de matriz, y su tamaño estará especificado en el archivo de propiedades mencionado anteriormente. Cada celda de la matriz contendrá la siguiente información: el identificador de la semilla plantada, si se ha regado ese día y el número de días que lleva plantada. Por ejemplo, un huerto con dos filas y tres columnas, donde la columna 1 está vacía y las columnas 0 y 2 contienen cultivos plantados, tendría el siguiente aspecto:

[20, true, 2]	[-1, false, -1]	[5, true, 3]
[20, true, 2]	[-1, false, -1]	[5, true, 3]

La particularidad del Huerto es que no se gestionará como una estructura de datos en memoria, sino que se almacenará en un **Fichero de Acceso Aleatorio** denominado “**huerto.dat**”, sobre el que se llevarán a cabo todas las operaciones: plantar cultivos, regar, cosechar, etc. Es importante destacar que los archivos de acceso aleatorio no comprenden la estructura de matriz, pero podemos simular su comportamiento desplazando el puntero a la celda deseada. Por ejemplo, para acceder a la celda [1, 0] es necesario omitir toda la fila 0. Para ello, debemos mover el puntero la cantidad de posiciones que corresponda al número de celdas en la fila 0, teniendo en cuenta el tamaño en bytes de cada celda.

Cada vez que se cierre el juego, es esencial guardar su estado en un archivo binario denominado “stardam_valley.bin”. De este modo, el jugador podrá reanudar su progreso en futuras sesiones de la aplicación, si así lo desea, garantizando una experiencia de juego continua y sin interrupciones.

2.1. Requisito funcional - Inicio de la aplicación

Al iniciar la aplicación, se debe verificar si existe un estado guardado. Si es así, se presentará un menú con dos opciones:

BIENVENIDO A STARDAM VALLEY

-
1. NUEVA PARTIDA
 2. CARGAR PARTIDA

En caso de que no exista un estado previo guardado solo se ofrecerá la opción de iniciar una nueva partida.

2.1.1. Nueva partida

Si ya existía una partida previa, se eliminará toda la información guardada de la misma. Luego, se permitirá al usuario personalizar la configuración de la partida, la cual se almacenará en un fichero de propiedades. Si el usuario no desea personalizar la configuración, se aplicarán los siguientes valores por defecto:

- Número de filas del huerto: 4
- Número de columnas del huerto: 4
- Presupuesto inicial: 1000
- Estación inicial: PRIMAVERA
- Días de duración de cada estación: 30

Debe existir siempre un archivo llamado `default_config.properties` con los valores expresados anteriormente. Si el usuario opta por personalizar la configuración, se generará un nuevo archivo llamado `personalized_config.properties` con los valores especificados por él.

Una vez configurado el archivo de propiedades, se procederá a inicializar todos los componentes del juego según los parámetros establecidos. Esto incluye la creación del archivo RAF del huerto, la inicialización de cada celda con los valores predeterminados (-1 para los enteros y false para los booleanos), y la generación de la tienda de semillas

para el día (ver la opción del menú "INICIAR NUEVO DÍA" para más información), entre otros elementos. Finalmente, se mostrará el menú de opciones y dará comienzo el juego.

2.1.2. Cargar partida

Esta opción solo aparecerá si se detecta que ya existe una partida guardada entre los archivos del juego. Su función será cargar el estado guardado y mostrar el menú principal del juego.

2.2. Requisito funcional - Programa principal

El juego debe tener las siguientes funcionalidades:

STARDAM VALLEY

1. INICIAR NUEVO DÍA
2. ATENDER CULTIVOS
3. PLANTAR CULTIVOS EN COLUMNA
4. VENDER COSECHA
5. MOSTRAR INFORMACIÓN DE LA GRANJA
6. SALIR

- La opción **INICIAR NUEVO DÍA** realiza las acciones necesarias para avanzar un día en el juego. Primero, actualiza el contador de días y verifica si es necesario cambiar de estación. En caso de cambio de estación, se eliminarán todos los cultivos existentes en el huerto, restableciendo cada celda a sus valores por defecto (-1 para los enteros y false para los booleanos). Si no hay cambio de estación, se analizarán los cultivos actuales: aquellos que fueron regados el día anterior actualizarán su contador de días plantados y volverán a establecerse como no regados. Ejemplo:

[20, true , 2]	[-1, false, -1]	[5, false, 3]	NUEVO DÍA →	[20, false , 3]	[-1, false, -1]	[5, false, 3]
[20, true , 2]	[-1, false, -1]	[5, false, 3]		[20, false , 3]	[-1, false, -1]	[5, false, 3]

Además, se generará la tienda del día. De las semillas que se pueden cultivar en la temporada actual juego, se seleccionarán tres variedades diferentes, que serán las que estén a la venta ese día.

2. La opción **ATENDER CULTIVOS** regará todas las celdas del huerto que tengan un cultivo plantado, omitiendo aquellas que estén vacías. Además, si alguna celda está lista para la cosecha, es decir, si el número de días que lleva plantado coincide con el tiempo necesario para que la semilla crezca, se procederá a su cosecha. El número máximo de frutos que puede producir cada cultivo está especificado en el campo maxFrutos del archivo semillas.xml. Una vez cosechado el cultivo, los valores de esa celda se restablecerán a sus valores por defecto. Todos los cultivos cosechados, junto con la cantidad recolectada, se deben guardar en el almacén. Ejemplo de salida de la opción ATENDER CULTIVOS, teniendo en cuenta que el trigo (id 5) tarda 4 días en cultivarse y como máximo genera 7 frutos cada semilla.

[20, false, 2]	[-1, false, -1]	[5, false, 4]	ATENDER CULTIVOS →	[20, true, 2]	[-1, false, -1]	[-1, false, -1]
[20, false, 2]	[-1, false, -1]	[5, false, 4]		[20, true, 2]	[-1, false, -1]	[-1, false, -1]

Salida por consola: *Se han añadido al almacén 10 frutos de trigo.*

3. La opción **PLANTAR CULTIVOS EN COLUMNA** permite al usuario plantar toda la columna seleccionada con la semilla de su elección, escogida entre las semillas disponibles en la tienda ese día. Solo se mostrarán las semillas para las que el usuario tenga suficiente presupuesto para comprarlas y no se podrá plantar en una columna que ya tenga cultivos o que no exista. Por ejemplo, plantamos fresas (id 11) en la columna 1:

[20, true, 2]	[-1, false, -1]	[5, false, 3]	PLANTAR →	[20, false, 3]	[11, false, 1]	[5, false, 3]
[20, true, 2]	[-1, false, -1]	[5, false, 3]		[20, false, 3]	[11, false, 1]	[5, false, 3]

4. La opción **VENDER FRUTOS** vende todos los productos que se encuentran en el almacén. El precio de venta de cada fruto está definido en el campo precioVentaFruto de semillas.xml.

Salida por consola:

Se han vendido 5 unidades de trigo por 75 euros.

Se han vendido 10 unidades de calabaza por 1500 euros.

Ganancias totales: 1575 euros.

5. La opción **MOSTRAR INFORMACIÓN DE LA GRANJA** debe mostrar, al menos, la siguiente información:

INFORMACIÓN DE LA GRANJA:

- Día de juego:6
- Dinero disponible: 280
- Estación: PRIMAVERA
- Semillas en venta: Patata Coliflor Ajo
- Frutos en almacén: [Ajo - 6]
- Estado del huerto:

[11-T-6][13-T-1][SS][SS]

[11-T-6][13-T-1][SS][SS]

[11-T-6][13-T-1][SS][SS]

[11-T-6][13-T-1][SS][SS]

NOTA: Al mostrar cada celda del huerto se ha optado por la nomenclatura “SS” (Sin Semilla) para indicar que dicha celda está vacía.

6. La opción **SALIR** guarda el estado del juego en el fichero StardamValley.bin y finaliza la ejecución del programa.

3. CONSEJOS Y CONSIDERACIONES

Puede parecer difícil porque hay muchos requisitos y varias opciones, **pero no es tan difícil como parece**. Antes de empezar a programar **lee bien el enunciado**, subraya lo importante, hazte notas, escribe un borrador de las posibles funciones, etc. Intenta entender el problema y cómo realizarlo antes de empezar a escribir código.

No intentes hacerlo todo de golpe. Una estrategia clave en programación es dividir un problema grande en varios problemas pequeños y luego resolverlos uno a uno. Empieza programando y probando las etapas una a una, añadiendo la siguiente cuando te asegures que la anterior funcione. Así será más fácil.

4. SOBRE EL PLAGIO O USO DE IA

Este trabajo está destinado a evaluar el nivel de adquisición de conocimientos de los alumnos de manera individual. Por lo tanto, la detección de material no original (plagio o copia) no permitirá dicha evaluación y acarreará una calificación de 0 (cero). Por los mismos motivos, la detección del uso de Inteligencia Artificial para la realización del trabajo acarreará una calificación de 0 (cero).

5. ENTREGA

Se deberá entregar el proyecto de IntelliJ en un archivo comprimido con el nombre del alumno. Para ello deberás usar la opción **FILE-> EXPORT -> PROJECT TO ZIP FILE**.

Además deberás entregar una pequeña memoria en la que indiques cuál ha sido tu estrategia para desarrollar el trabajo, los principales retos que se te han planteado y cómo los has resuelto. Incluye también un comentario sobre el nivel de dificultad de la tarea y si crees que te ha servido para aplicar/afianzar conocimientos.

6. RÚBRICA DE CALIFICACIÓN

Aspecto a Evaluar	Nivel Insuficiente [0]	Nivel Básico [4]	Nivel Competente [7]	Nivel Sobresaliente [10]	Peso sobre el total
1. Claridad, Organización y Separación de responsabilidades. Uso adecuado de la Programación Orientada a Objetos. (Es necesaria una calificación ≥ 4 para evaluar el resto de aspectos)	El código es confuso, desorganizado y difícil de seguir. El diseño de clases no es correcto y/o asumen responsabilidades que no les pertenecen.	El código es legible pero podría estar mejor organizado. El diseño de clases presenta numerosos errores y/o la mayoría de clases asumen responsabilidades que no les pertenecen.	El código está bien organizado y es fácil de leer. El diseño de clases es correcto en su mayoría y/o alguna clase asume responsabilidades que no le pertenecen. No se ha hecho uso del patrón Singleton.	El código está excepcionalmente organizado y fácil de entender. El diseño de clases es excelente y las clases tienen una correcta separación de responsabilidades. Se ha hecho uso del patrón Singleton para el manejo de algún fichero.	10%
2. Requisito funcional: Inicio de la aplicación (Es necesaria una calificación ≥ 4 para evaluar el resto de aspectos)	No se implementa el requisito o la implementación no es correcta: escritura fichero RAF, lectura/escritura fichero propiedades y/o fichero binario.	Hay fallos significativos en la implementación del requisito.	Se implementa el requisito pero podría haber fallos menores.	La implementación cumple con todos los requisitos especificados.	15%
3. Requisito funcional: iniciar nuevo día	No se implementa y/o el código presenta numerosos errores. El tratamiento del huerto no es correcto.	Hay fallos significativos en la implementación del requisito.	Se implementa el requisito pero podría haber fallos menores. El fichero del huerto se podría gestionar de manera más eficiente.	La implementación cumple con todos los requisitos especificados. El fichero del huerto se gestiona de la manera más eficiente posible.	20%

4. Requisito funcional: atender cultivos	No se implementa y/o el código presenta numerosos errores. El tratamiento del huerto no es correcto.	Hay fallos significativos en la implementación del requisito.	Se implementa el requisito pero podría haber fallos menores. El fichero del huerto se podría gestionar de manera más eficiente.	La implementación cumple con todos los requisitos especificados. El fichero del huerto se gestiona de la manera más eficiente posible.	20%
5. Requisito funcional: plantar cultivos en columna	No se implementa y/o el código presenta numerosos errores. El tratamiento del huerto no es correcto.	Hay fallos significativos en la implementación del requisito.	Se implementa el requisito pero podría haber fallos menores. El fichero del huerto se podría gestionar de manera más eficiente.	La implementación cumple con todos los requisitos especificados. El fichero del huerto se gestiona de la manera más eficiente posible.	20%
6. Requisito funcional: vender cosecha	No se implementa y/o el código presenta numerosos errores.	Hay fallos significativos en la implementación del requisito.	Se implementa el requisito pero podría haber fallos menores.	La implementación cumple con todos los requisitos especificados.	5%
7. Requisito funcional: mostrar granja	No se implementa y/o el código presenta numerosos errores.	Hay fallos significativos en la implementación del requisito.	Se implementa el requisito pero podría haber fallos menores.	La implementación cumple con todos los requisitos especificados.	5%
8. Originalidad	El código no muestra intentos de creatividad ni mejora más allá de los requisitos básicos.	El código muestra algunas mejoras o enfoques creativos adicionales.	El código demuestra un esfuerzo significativo para mejorar y/o una creatividad notable.	El código va más allá de las expectativas y demuestra una creatividad excepciones y mejoras innovadoras.	5%